

Trabajo Práctico Integrador

Programación 2

Alumnos: Martin Sisterna y Benjamin Arrollo

**18 de Junio de 2026, Universidad
Tecnológica Nacional**

Índice

Marco Teórico	3, 4, 5, 6
• UML	3
• Java	4
• Bases de datos relacionales	5
• Persistencia con JDBC	6
Decisiones Técnicas y Arquitectura	7 a 12
Dificultades y Soluciones	12, 13
Bibliografía/Webgrafía	14

Marco teórico

Tecnologías utilizadas: UML, Java, base de datos relacionales, persistencia con JDBC, Manejo de Errores

UML

(Lenguaje Unificado de Modelado)

Imagínate que vas a construir un rascacielos: no empezarías a poner ladrillos y a verter hormigón sin antes tener unos planos detallados creados por un arquitecto, ¿verdad? En el desarrollo de software ocurre lo mismo. Antes de escribir miles de líneas de código, se utiliza UML para diseñar la estructura del sistema.

UML no es un lenguaje de programación como Python, Java o C++, sino un lenguaje visual. Es un estándar internacional que utiliza un conjunto de símbolos, reglas y diagramas para representar gráficamente cómo está estructurado y cómo funciona un sistema de software.

Fue creado en la década de 1990 por tres grandes ingenieros de software (Grady Booch, James Rumbaugh e Ivar Jacobson) con el objetivo de unificar los diferentes métodos de modelado que existían en la época. Hoy en día, es el estándar universal de la industria.

UML cumple varias funciones críticas dentro del ciclo de vida del desarrollo de software:

Visualizar: Permite ver de forma gráfica cómo se relacionan los diferentes componentes de un sistema, lo que facilita la comprensión de arquitecturas complejas que serían difíciles de entender leyendo solo código fuente.

Especificar: Ayuda a definir con precisión las características, funcionalidades y requisitos de un sistema antes de empezar a programarlo.

Documentar: Funciona como la documentación oficial del proyecto. Si un nuevo programador se une al equipo, mirar los diagramas UML le permitirá entender el sistema en cuestión de horas en lugar de semanas.

Planificar: Facilita la comunicación entre los clientes (que explican lo que necesitan) y los desarrolladores (que traducen esa necesidad en lógica técnica).

En pocas palabras UML sirve como un puente de comunicación. Permite que los desarrolladores no se pierdan en la complejidad de su propio código y que los clientes entiendan la estructura de lo que están comprando, todo a través de dibujos técnicos estandarizados.

JAVA

Java es uno de los lenguajes de programación más importantes, influyentes y utilizados del mundo. Nacido a mediados de los años 90 de la mano de Sun Microsystems, el cual hoy en día es propiedad de Oracle, su creación revolucionó la informática gracias a una filosofía muy clara: "Escribe una vez, ejecútalo en cualquier lugar"

Java es un lenguaje de programación de alto nivel, orientado a objetos y plattformamente independiente.

A diferencia de otros lenguajes de la época como C o C++, que debían ser compilados específicamente para el sistema operativo en el que se iban a ejecutar como Windows o Linux, Java introdujo un componente revolucionario: la Máquina Virtual de Java (JVM).

Java está en todas partes. Desde aplicaciones médicas hasta consolas de videojuegos, su versatilidad lo ha mantenido en el top de la industria por décadas. Sus usos principales son:

Aplicaciones Móviles (Android): Durante años, Java fue el lenguaje oficial y exclusivo para crear aplicaciones de Android. Aunque hoy comparte protagonismo con Kotlin, la inmensa mayoría del ecosistema Android sigue basándose en Java.

Sistemas Empresariales (Backend): Es el rey indiscutible en los servidores de grandes bancos, aerolíneas, compañías de seguros y gobiernos. Su robustez y seguridad lo hacen ideal para manejar volúmenes gigantescos de datos y transacciones críticas.

Desarrollo Web: A través de tecnologías como Spring Boot o Jakarta EE, se utiliza para construir la lógica interna (el backend) de plataformas web masivas.

Internet de las Cosas (IoT): Se usa en chips de tarjetas de crédito, televisores inteligentes, electrodomésticos y sistemas de navegación automatizada.

Videojuegos: El caso más famoso del mundo es Minecraft, cuya versión original para PC fue desarrollada completamente en Java.

Actualmente Java sigue siendo único y popular gracias a su estructura orientada a objetos, que permite organizar proyectos masivos de forma limpia y reusable, y a su gestión automática de memoria (Garbage Collector), que optimiza el rendimiento del sistema sin intervención del programador. Además, destaca por su alto nivel de seguridad contra accesos corruptos y por contar con una comunidad global gigante, lo que garantiza soporte inmediato y soluciones documentadas para cualquier tipo de error.

BASES DE DATOS RELACIONALES

Una Base de Datos Relacional es un sistema para almacenar y organizar información de forma estructurada, utilizando tablas compuestas por filas y columnas que se conectan entre sí mediante relaciones lógicas.

Este modelo, ideado en 1970 por Edgar F. Codd, se basa en la idea de que los datos de la vida real tienen conexiones entre sí. Por ejemplo, en una tienda online, la tabla de Clientes se relaciona con la tabla de Pedidos, y esta a su vez con la tabla de Productos. La gran ventaja de este sistema es que evita la duplicación de información y garantiza la consistencia de los datos a través de reglas estrictas. Para interactuar con estas bases de datos (crear tablas, consultar información o borrar registros), se utiliza un lenguaje estándar universal llamado SQL (Structured Query Language).

Las bases de datos relacionales son el motor invisible de casi cualquier aplicación moderna que requiera precisión matemática y seguridad en sus transacciones. Se utilizan principalmente para:

Sistemas Financieros y Bancarios: Donde perder un registro o duplicar un saldo sería catastrófico.

Plataformas de Comercio Electrónico: Para gestionar inventarios, procesar compras y asociar carritos con usuarios específicos.

Sistemas de Facturación y Logística: Donde se necesita rastrear qué cliente compró qué producto y a través de qué repartidor.

Actualmente los sistemas de gestión de bases de datos relacionales o por sus siglas RDBMS más conocidos en la industria son MySQL, PostgreSQL, Oracle Database y Microsoft SQL Server.

Persistencia con JDBC 4,5

La persistencia de datos es la capacidad de una aplicación para guardar sus datos (los objetos que están en la memoria RAM) en un almacenamiento permanente como por ejemplo en una base de datos relacional para que no se pierdan al apagar el programa.

En el ecosistema Java, la herramienta fundamental y nativa para lograr esto es **JDBC (Java Database Connectivity)**. Su especificación más reciente, **JDBC 4.5**, está diseñada para trabajar con las versiones modernas de Java, optimizando la comunicación y añadiendo soporte para tipos de datos contemporáneos.

JDBC es una API (un conjunto de clases e interfaces en los paquetes `java.sql` y `javax.sql`) que actúa como un **traductor universal**. Permite que una aplicación Java se conecte a prácticamente cualquier base de datos relacional (como MySQL, PostgreSQL u Oracle) utilizando el mismo código básico.

Sin JDBC, tendrías que escribir un código completamente diferente si tu aplicación usa MySQL y luego decides cambiar a Oracle. Con JDBC, solo cambias el "Driver" (el controlador específico del proveedor) y el resto de tu código Java permanece intacto.

Para persistir o consultar datos con JDBC, el código sigue una serie de pasos fijos

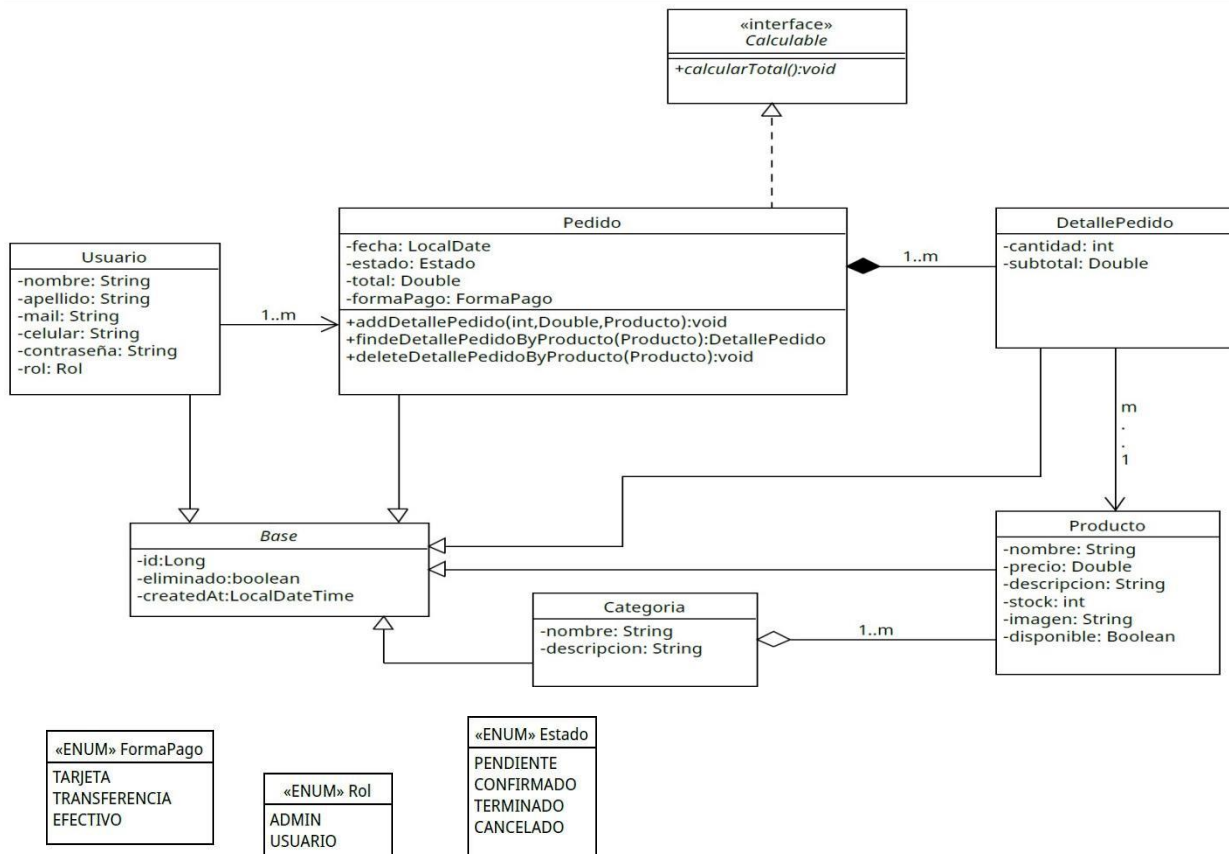
- 1- **Establecer la Conexión:** Se solicita una conexión a la base de datos mediante una URL de conexión y credenciales de usuario.
- 2- **Crear la Sentencia (Statement):** Se prepara la consulta SQL que se desea ejecutar. Para evitar hackeos (Inyección SQL), se usan `PreparedStatement`.
- 3- **Ejecutar y Procesar:** Se envía el SQL a la base de datos. Si es una consulta, devuelve un `ResultSet` (una estructura de filas y columnas) que el programador recorre para mapear los datos de vuelta a objetos Java.
- 4- **Cerrar Recursos:** Se deben cerrar la conexión y las sentencias para no saturar el servidor.

Aunque hoy en día muchos desarrolladores utilizan herramientas de mayor nivel como Hibernate o Spring Data JPA para no escribir SQL a mano, todos ellos utilizan JDBC por detrás. Conocer JDBC 4.5 es entender cómo se mueven realmente los datos entre Java y tu base de datos a la máxima velocidad posible.

Decisiones Técnicas y Arquitectura

La resolución del Trabajo Practico Integrador se resolvió de la siguiente manera:

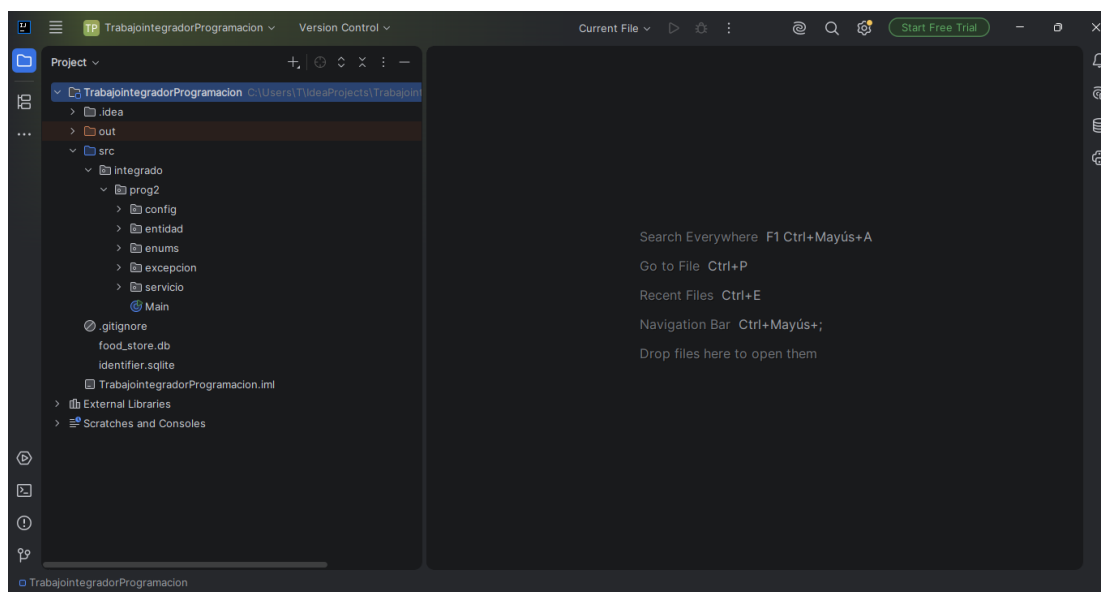
Primero después de analizar el modelo UML y las entidades proporcionadas nos dimos cuenta que nos servían y nos sentíamos cómodos con esa metodología para el trabajo entonces la utilizamos y implementamos



El editor de código que utilizamos para trabajar en este proyecto fue IntelliJ IDEA el cual creamos los siguientes paquetes, la arquitectura elegida fue la siguiente:

```

src/
├── integrado/
│   ├── prog2/
│   │   ├── Main.java          <-- Punto de entrada e interfaz de usuario
│   │   ├── config/           <-- Configuraciones del sistema
│   │   ├── entities/         <-- Clases del modelo UML (dominio)
│   │   ├── enums/           <-- Enumeraciones (Rol, Estado, FormaPago)
│   │   ├── exception/       <-- Excepciones personalizadas
│   │   └── services/        <-- Lógica de negocio y gestión de colecciones
  
```



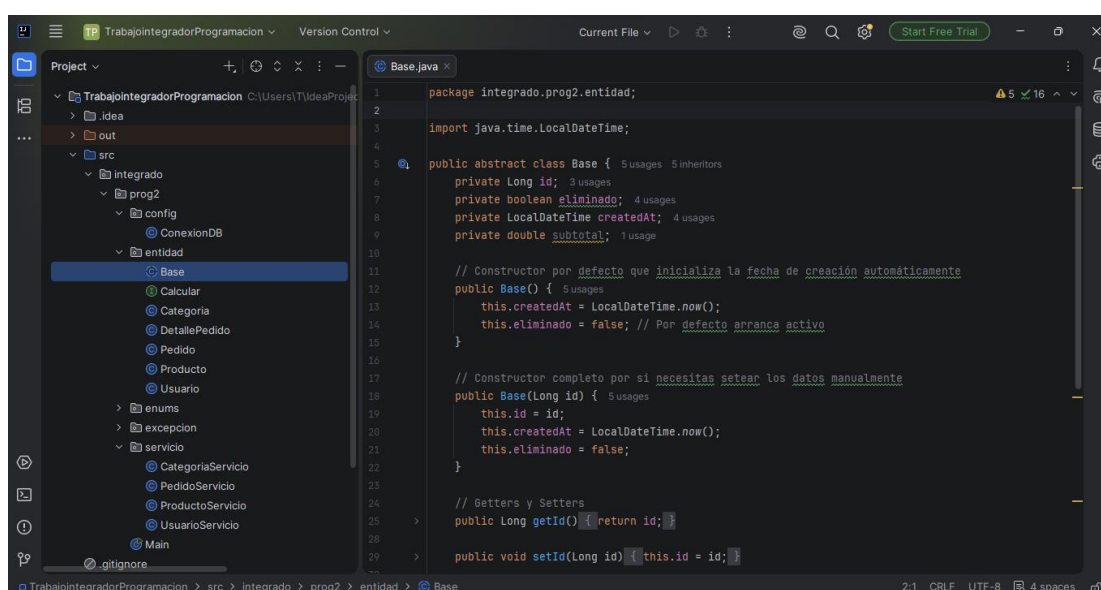
Acto seguido para organizar el código, definimos una estructura de paquetes lógica que separa las responsabilidades de cada componente:

Entidad: Contiene el modelo de dominio basado en el diagrama UML, donde cada clase hereda de una clase base común para asegurar la trazabilidad mediante los atributos id, eliminado y createdAt.

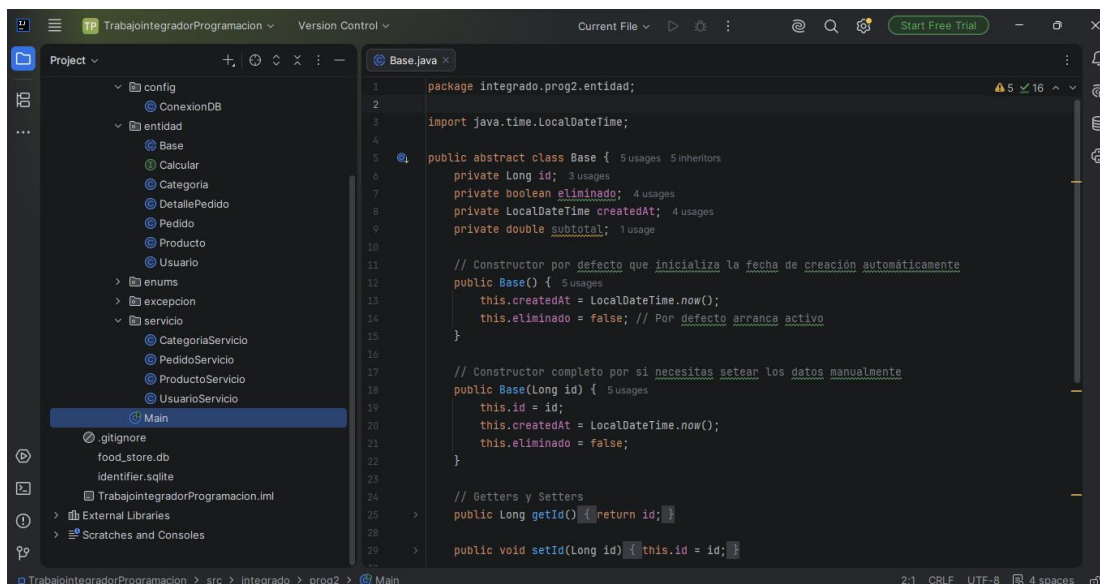
Servicios: Coordina las reglas de negocio, validaciones y la comunicación con la capa de persistencia.

Config: Gestiona la conexión a la base de datos, garantizando que el entorno esté preparado para operar al iniciar la aplicación.

Main: Actúa como el punto de entrada, manejando la interacción con el usuario mediante un menú de consola basado en un bucle do-while.



Para desarrollar el sistema, decidimos organizar el código en "paquetes" o carpetas. Esta estructura nos ayudó mucho a mantener el orden y a saber exactamente dónde estaba cada parte del programa: los datos (entidades), la lógica que hace funcionar todo (servicios) y la comunicación con la base de datos.



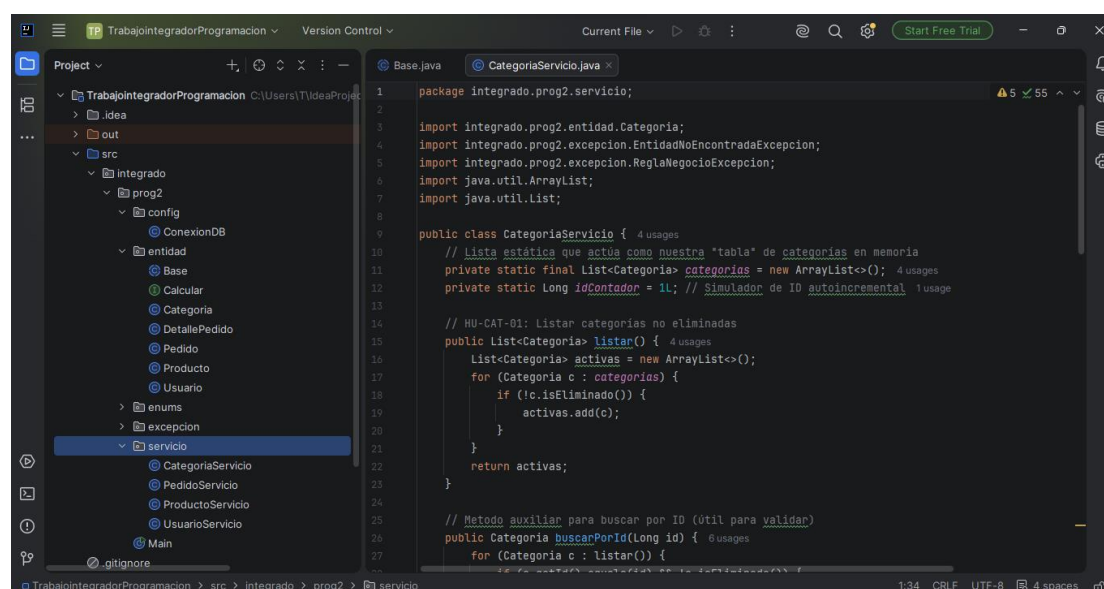
The screenshot shows an IDE with the project structure on the left and the `Base.java` file open in the editor. The project structure includes:

- config
- ConexionDB
- entidad
 - Base
 - Calcular
 - Categoria
 - DetallePedido
 - Pedido
 - Producto
 - Usuario
- enums
- excepcion
- servicio
 - CategoriaServicio
 - PedidoServicio
 - ProductoServicio
 - UsuarioServicio
- Main

The `Base.java` file contains the following code:

```
1 package integrado.prog2.entidad;
2
3 import java.time.LocalDateTime;
4
5 public abstract class Base {
6     private Long id;
7     private boolean eliminado;
8     private LocalDateTime createdAt;
9     private double subtotal;
10
11     // Constructor por defecto que inicializa la fecha de creación automáticamente
12     public Base() {
13         this.createdAt = LocalDateTime.now();
14         this.eliminado = false; // Por defecto arranca activo
15     }
16
17     // Constructor completo por si necesitas setear los datos manualmente
18     public Base(Long id) {
19         this.id = id;
20         this.createdAt = LocalDateTime.now();
21         this.eliminado = false;
22     }
23
24     // Getters y Setters
25     public Long getId() { return id; }
26
27     public void setId(Long id) { this.id = id; }
```

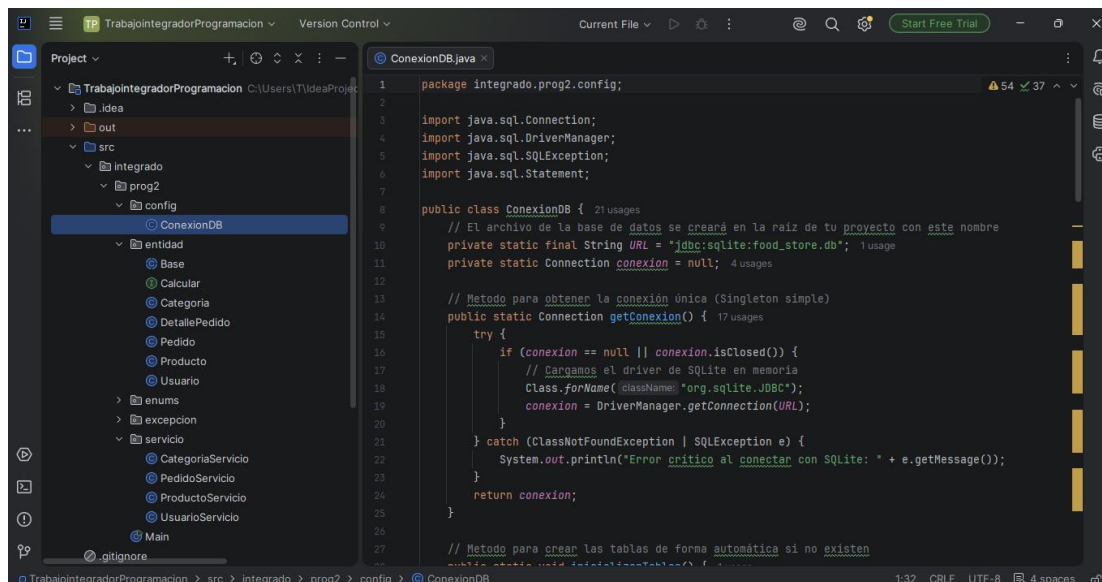
En cuanto a cómo conectamos la base de datos, aunque el diseño original estaba pensado para ser muy modular, al momento de programar decidimos integrar el acceso a los datos directamente dentro de las clases de servicio.



The screenshot shows the same IDE with the project structure on the left and the `CategoriaServicio.java` file open in the editor. The project structure is the same as in the previous screenshot. The `CategoriaServicio.java` file contains the following code:

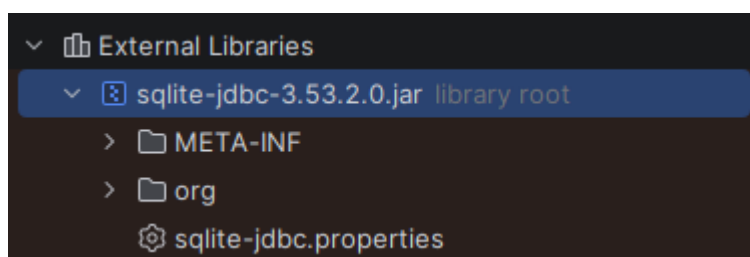
```
1 package integrado.prog2.servicio;
2
3 import integrado.prog2.entidad.Categoria;
4 import integrado.prog2.excepcion.EntidadNoEncontradaExcepcion;
5 import integrado.prog2.excepcion.ReglaNegocioExcepcion;
6 import java.util.ArrayList;
7 import java.util.List;
8
9 public class CategoriaServicio {
10     // Lista estática que actúa como nuestra "tabla" de categorías en memoria
11     private static final List<Categoria> categorias = new ArrayList<>();
12     private static Long idContador = 1L; // Simulador de ID autoincremental
13
14     // HU-CAT-01: Listar categorías no eliminadas
15     public List<Categoria> listar() {
16         List<Categoria> activas = new ArrayList<>();
17         for (Categoria c : categorias) {
18             if (!c.isEliminado()) {
19                 activas.add(c);
20             }
21         }
22         return activas;
23     }
24
25     // Metodo auxiliar para buscar por ID (útil para validar)
26     public Categoria buscarPorId(Long id) {
27         for (Categoria c : listar()) {
28             if (c.getId().equals(id)) {
29                 return c;
30             }
31         }
32         return null;
33     }
34 }
```

Si bien existen formas más complejas de hacerlo (como el patrón DAO), optamos por este camino porque nos permitió ser más prácticos, tener menos archivos dando vueltas y entender mejor cómo fluye la información desde la base de datos hasta la pantalla. Esto nos ayudó a detectar errores mucho más rápido y a que el código fuera más fácil de seguir y mantener durante todo el proceso.

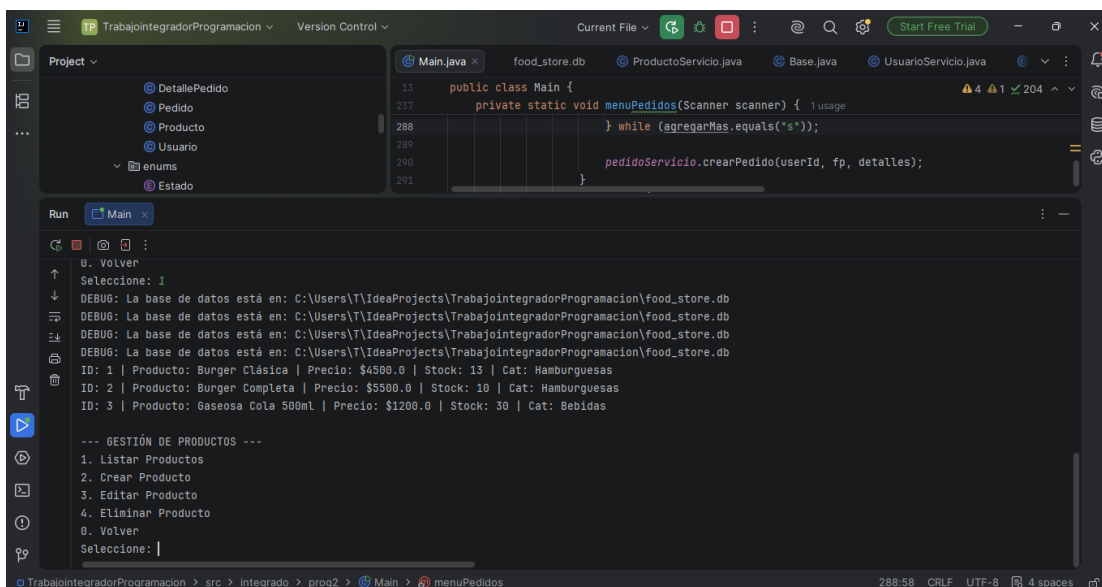


Aunque inicialmente la consigna sugería el almacenamiento en memoria, tomamos la decisión técnica de implementar JDBC con SQLite. Optamos por esta tecnología ya que permite persistir la información en un archivo de base de datos generado automáticamente por el sistema al inicializar.

Para conectar el programa con la base de datos, usamos una herramienta llamada driver JDBC de SQLite. Su función es tomar las instrucciones que escribimos en Java y pasarlas al lenguaje que la base de datos entiende (SQL). Gracias a esto, logramos que la información que cargamos en el sistema no se pierda al cerrar el programa, dándonos así una forma segura y permanente de guardarlo en un archivo propio.



El listado de un producto



```

public class Main {
    private static void menuPedidos(Scanner scanner) {
        while (agregarMas.equals("s"));
        pedidoServicio.crearPedido(userId, fp, detalles);
    }
}

```

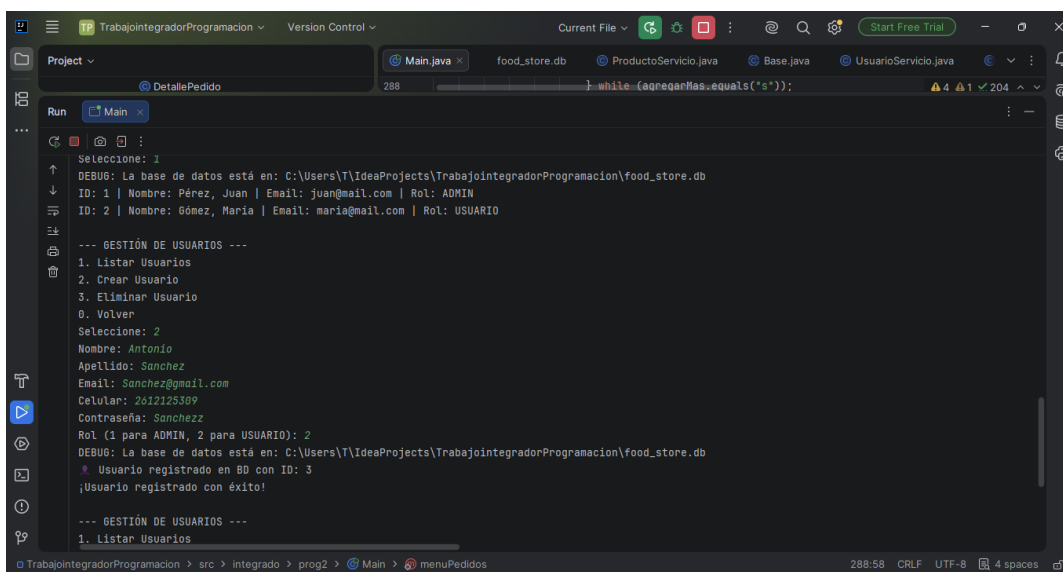
```

Seleccione: 1
DEBUG: La base de datos está en: C:\Users\T\IdeaProjects\TrabajointegradorProgramacion\food_store.db
DEBUG: La base de datos está en: C:\Users\T\IdeaProjects\TrabajointegradorProgramacion\food_store.db
DEBUG: La base de datos está en: C:\Users\T\IdeaProjects\TrabajointegradorProgramacion\food_store.db
ID: 1 | Producto: Burger Clásica | Precio: $4500.0 | Stock: 13 | Cat: Hamburguesas
ID: 2 | Producto: Burger Completa | Precio: $5500.0 | Stock: 10 | Cat: Hamburguesas
ID: 3 | Producto: Gaseosa Cola 500ml | Precio: $1200.0 | Stock: 30 | Cat: Bebidas

--- GESTIÓN DE PRODUCTOS ---
1. Listar Productos
2. Crear Producto
3. Editar Producto
4. Eliminar Producto
0. Volver
Seleccione:

```

Creación y listado de usuarios



```

Seleccione: 1
DEBUG: La base de datos está en: C:\Users\T\IdeaProjects\TrabajointegradorProgramacion\food_store.db
ID: 1 | Nombre: Pérez, Juan | Email: juan@mail.com | Rol: ADMIN
ID: 2 | Nombre: Gómez, María | Email: maria@mail.com | Rol: USUARIO

--- GESTIÓN DE USUARIOS ---
1. Listar Usuarios
2. Crear Usuario
3. Eliminar Usuario
0. Volver
Seleccione: 2
Nombre: Antonio
Apellido: Sanchez
Email: Sanchez@gmail.com
Celular: 2612125309
Contraseña: Sanchezz
Rol (1 para ADMIN, 2 para USUARIO): 2
DEBUG: La base de datos está en: C:\Users\T\IdeaProjects\TrabajointegradorProgramacion\food_store.db
* Usuario registrado en BD con ID: 3
¡Usuario registrado con éxito!

--- GESTIÓN DE USUARIOS ---
1. Listar Usuarios

```

Dificultades y Soluciones

Durante el desarrollo del proyecto surgieron varios obstáculos técnicos propios de integrar persistencia real en un sistema que originalmente fue pensado para trabajar en memoria. A continuación se detallan los principales problemas encontrados y cómo se resolvieron.

Configuración inicial de la conexión JDBC con SQLite:

La integración de SQLite mediante JDBC presentó un desafío inicial relacionado con la resolución de rutas de archivo en diferentes entornos de ejecución. Inicialmente, el uso de una ruta relativa generaba comportamientos inconsistentes, ya que la ubicación del archivo `food_store.db` dependía del directorio de trabajo (working directory) configurado por el IDE o la consola en cada sesión, provocando la dispersión de los datos y una aparente "pérdida" de información entre ejecuciones.

Transacciones en la creación de pedidos (rollback y control de stock)

Uno de los desafíos técnicos más críticos del sistema fue garantizar la integridad de los datos durante el registro de pedidos. Debido a que un pedido implica múltiples operaciones relacionadas (creación de la cabecera, inserción de múltiples detalles y actualización de stock de varios productos), se debía evitar a toda costa dejar la base de datos en un estado inconsistente ante una interrupción.

Para resolver esto, implementamos un control de **transacciones atómicas** siguiendo el principio **ACID** (Atomicidad, Consistencia, Aislamiento y Durabilidad):

Atomicidad (A): Se forzó la gestión manual de la transacción desactivando el modo de confirmación automática mediante `conn.setAutoCommit(false)`. Esto permite tratar a todo el proceso como una unidad de trabajo indivisible: *todo se confirma o nada se aplica*.

Validación Proactiva y Consistencia (C): El sistema realiza un ciclo de validación de disponibilidad de stock antes de iniciar cualquier escritura. Si un solo producto carece de inventario, el proceso se aborta preventivamente, protegiendo la integridad del catálogo.

Mecanismo de Rollback: Ante cualquier excepción durante el flujo (ya sea por error de conexión, violación de restricciones o falta de stock), el bloque `catch` invoca explícitamente el método `rollback()`. Esta instrucción revierte automáticamente cualquier modificación previa en la base de datos, garantizando que no existan "pedidos huérfanos" ni descuentos de stock parciales.

Commit Final: Únicamente al completar exitosamente todas las validaciones e inserciones, se ejecuta el método `commit()`, haciendo que los cambios sean permanentes.

Colecciones en memoria desincronizadas de la base de datos

Esta fue probablemente la dificultad que más tiempo nos llevó detectar. El diseño original de las entidades mantenía listas internas, como la lista de detalles dentro de la clase Pedido, que se actualizaban localmente con métodos como `addDetallePedido()`. Al incorporar la persistencia con JDBC, la lógica real de guardado pasó a vivir en las clases de servicio, que escriben directamente contra SQLite. El problema fue que varias de esas listas en memoria quedaron sin conectarse con lo que efectivamente se guardaba en la base de datos: se actualizaba un lado pero no el otro, y los objetos en memoria mostraban información (por ejemplo, totales o listados) que no coincidía con lo persistido. Esto generó errores difíciles de rastrear porque el programa no fallaba, simplemente mostraba datos inconsistentes. Pudimos identificarlo revisando con cuidado, paso a paso, cada punto donde se modificaba una colección en memoria, y verificando si existía su contraparte equivalente en una consulta SQL. La solución fue unificar el criterio: toda lectura de datos para mostrar en pantalla se obtiene siempre consultando directamente la base de datos en el momento, en lugar de confiar en listas que se mantenían aparte en memoria.

Bibliografía / Webgrafía

Oracle. (2026). *Java SE Documentation*. <https://docs.oracle.com/en/java/javase/>

Oracle. (2026). *Java Database Connectivity (JDBC) Tutorial*.
<https://docs.oracle.com/javase/tutorial/jdbc/>

SQLite. (2026). *Official SQLite Documentation*. <https://www.sqlite.org/docs.html>

Xerial. (2026). *SQLite JDBC Driver*. GitHub Repository. <https://github.com/xerial/sqlite-jdbc>

Object Management Group (OMG). (2026). *Unified Modeling Language (UML) Specification*.
<https://www.omg.org/spec/UML/>

Repositorio del proyecto (GitHub):
<https://github.com/MartinS7-hub/Trabajo-Integrador-ProgramacionII.git>